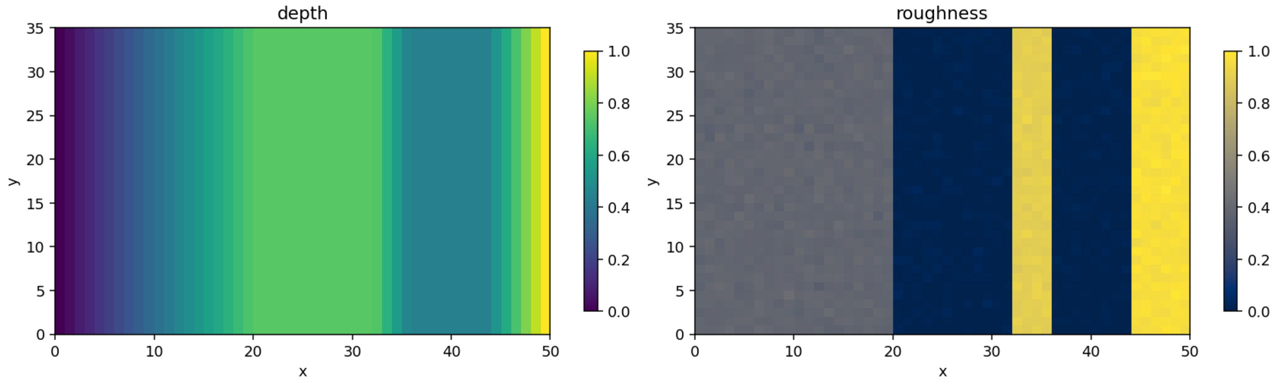
Spatial Bayesian search — project walkthrough

Bayesian Inference — Deliverable 2. This report walks through every modeling choice and engineering layer built in **gemini_proyect/**: priors, conditional detectability, likelihood and posterior update, search strategies, the local Streamlit replica of the search-missions webapp, and the multi-campaign benchmark used to compare strategies.

Grid: depth and roughness



The grid is fixed at N_x **= 50,** N_y **= 35**. Each cell carries a normalised depth in [0, 1] and a normalised roughness in [0, 1]. Roughness is bimodal (mostly ~0 or ~0.4); depth ramps from shallow (left) to deep (right).

1. Prior distribution (Task 1)

The accident notebook gives the explosion point x_E = (7, 20) and three physical vectors: v_{plane} = (6, –3.5), v_{wind} = (–1, –1.5), v_{drift} = (0.5, –1.5). Two qualitative witness statements add that the object kept moving forward along the plane trajectory and fell slightly off to one side.

Modelling choice

We treat the expected seabed location as a deterministic displacement of x_E:

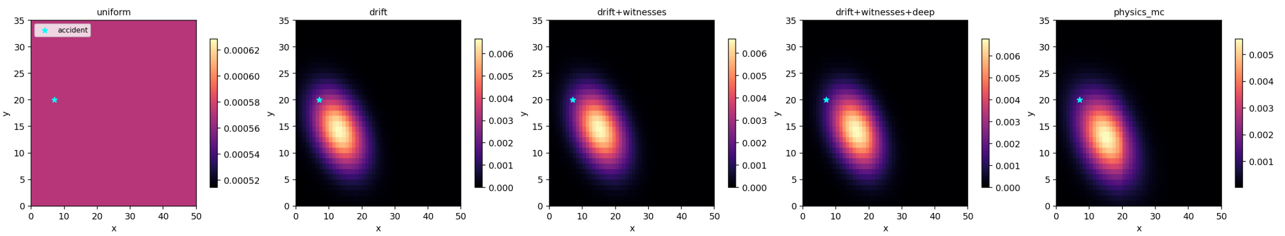
$$\mu = x_E + \tau_{fall} \cdot (v_{plane} + w_{wind} \cdot v_{wind}) + \tau_{drift} \cdot v_{drift}$$

and place an **anisotropic 2-D Gaussian** around μ whose standard deviations along/perpendicular to the trajectory are $\sigma_{\blacksquare}$ = 6 and $\sigma_{\perp}$ = 4. The along-trajectory uncertainty is larger because we don't know how far the object actually travelled. The witness shifts (forward bias, lateral offset of ~1.5 grid units) are smaller than the spreads so they behave as soft hints, not facts.

Five candidate priors implemented

uniform (baseline), **drift** (physics only with deterministic constants), **drift+witnesses** (adds the two witness shifts), **drift+witnesses+deep** (multiplied by exp(depth_bias · depth) to encode the soft hypothesis that a sinking object ends up in deeper cells), and **physics_mc** (Monte Carlo marginalisation over the uncertain physics constants τ_{fall}, w_{wind}, τ_{drift}, the two ellipse σ's, and the witness shifts — plus depth and roughness factors and a small uniform component so the prior is never literally zero anywhere).

Why physics_mc. The deterministic drift Gaussian is implausibly narrow: we do not know τ_{fall} to better than ~50%, the wind coupling is unconstrained, and witness 2 says "slightly off to one side" without specifying which side. Sampling those nuisance parameters and averaging the resulting Gaussians gives a wider, multi-modal prior that respects how little we actually know — the standard trick of Bayesian search theory (Stone et al., used in the Scorpion, Air France 447 and MH370 searches).



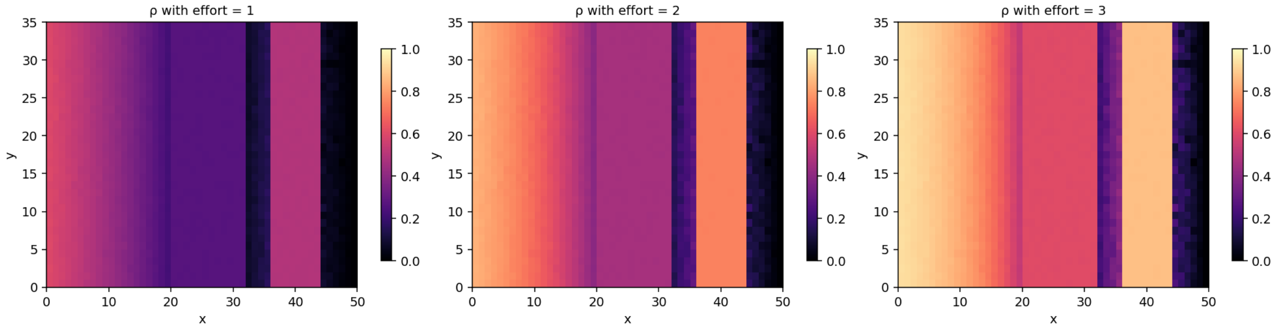
Each prior sums to 1 and is positive everywhere. The accident point (cyan star) is on the left of every panel; the prior mass shifts toward (x ≈ 13, y ≈ 14) once the physics is applied.

2. Conditional detectability $\rho_{t,j}$ (Task 2)

Per the deliverable, the detection probability for mission t, given the object is in cell j, must factorise as $\mathbf{q_{t,j} = c_{t,j}} \cdot \rho_{\mathbf{t,j}}$, where $c_{t,j} \in \{0, 1\}$ is the coverage indicator and $\rho_{t,j}$ is what we need to model.

$$\rho(d, r, e) = 1 - \exp(-\lambda_0 \cdot (1-d)^{a_d} \cdot (1-r)^{a_r} \cdot e)$$

Why this form. (i) $\rho \in [0, 1)$ for any parameters. (ii) Effort e enters multiplicatively in the rate, so e=2 is the survival function of two independent sensor passes — matches Mission Report 4, which needed repeated inspection. (iii) (1–d), (1–r) shrink the rate in deep/rough cells — matches Reports 1 and 3 which described low detectability in moderately deep / highly irregular terrain. (iv) Parameters are interpretable: λ_0 is the per-effort detection rate in the easiest possible cell.



ρ for effort levels 1, 2, 3 on the fixed grid. Shallow + smooth cells (top-left, dark-purple band) are easy at any effort; deep + rough cells (right side) need effort 3 just to reach ρ ≈ 0.5.

3. Likelihood and posterior update (Task 3)

Likelihood from a single mission. Given the binary outcome $s_t \in \{0, 1\}$, the model says

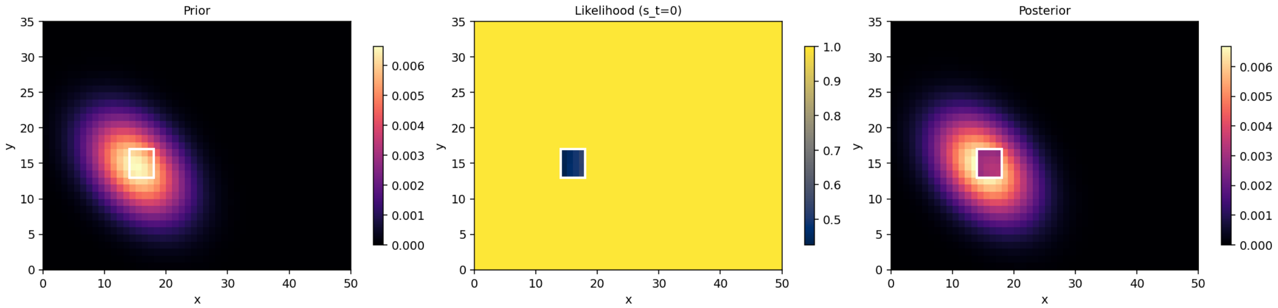
$$L_j(s_t) = q_{t,j}^{s_t} \cdot (1 - q_{t,j})^{1 - s_t}$$

If a cell is **not covered**, q = 0, so L = 1 when s = 0 (no information) and L = 0 when s = 1 (a detection rules out every uncovered cell — only covered cells could have produced the hit).

Full posterior across the history

Assuming conditional independence of detector outcomes given Z = j, the joint likelihood of the observed history is $\mathbf{L_j = \prod_t q_{t,j}^{s_t} (1 - q_{t,j})^{1 - s_t}}$, and the posterior is $\pi_j \cdot L_j$, normalised. We implement this in log-space to avoid underflow across long histories.

Worked example: one mission



Single mission illustration: rectangle x ∈ [14.5, 17.5], y ∈ [13.5, 16.5], effort 2, applied on the drift+witnesses prior. The likelihood panel shows which cells were compatible with the observed outcome; the posterior is the prior multiplied by that likelihood and renormalised.

Important corollary: before any mission

There is no "initial likelihood". With zero observations the posterior is exactly the prior. The likelihood is born with each mission and accumulates multiplicatively.

4. Search strategies (Task 4)

All strategies enumerate axis-aligned rectangles (the only thing the webapp accepts) of allowed widths and heights and score them. Scoring uses 2-D integral images so each call is ≈ 50 ms even with tens of thousands of candidates.

Cooldown reality check. The real webapp imposes a 12 h cooldown between submissions. Across a 230-coin budget the practical horizon is at most ~10 missions, so any policy that needs 50+ 1×1 probes (info_gain, max_expected_detection) is off the table. The two cooldown-friendly strategies are **cooldown_aware** (a few big high-effort sweeps planned against a target mission count) and **commit_and_verify** (stay in a zone until ruled out at a chosen confidence level).

cooldown_aware

Designed for the 12 h cooldown regime: a per-mission budget soft cap of *budget_remaining / n_missions_target*, a minimum rectangle size, effort $\in$ {2, 3} only. Among feasible rectangles it maximises raw expected detection probability $\Sigma_j \pi_j \cdot \rho(j, e)$ — not per cost, because cost is already controlled by the size floor and effort range. Result: 3–7 large missions per campaign instead of 50+ tiny ones.

max_expected_detection

argmax of $\Sigma_j \pi_j \cdot q_{t,j}$ / cost. Greedy: each call picks the rectangle that maximises expected detection probability per coin spent. Simple, sample-efficient, but tends to drop hard cells too fast. Kept for benchmarking only; not viable under the cooldown.

info_gain

argmax of expected entropy reduction per cost: $E[H(prior) – H(post \mid s_t)]$ / cost. One-step Bayesian optimal experiment. Explores broader than max_expected_detection because it values disambiguation, not just detection.

max_posterior_rect

argmax of $\Sigma_j \pi_j \cdot q_{t,j}$ (no cost normalisation). Prefers large rectangles when the posterior is concentrated — a small number of expensive sweeps.

commit_and_verify

Stateful: chooses an initial rectangle R whose posterior mass ≥ 1 – confidence and whose cell count ≥ *min_cells*, then *keeps probing R* until $P(Z \in R \mid data)$ drops below 1 – confidence (zone ruled out) or $s_t = 1$. Prevents premature abandonment of deep / rough cells where a single miss is weak evidence. With effort $\in$ {2, 3} and a 3×3 size floor it averages 2–4 missions per campaign.

thompson

Posterior sampling. Each call draws $j^* \sim \pi$ and then picks the best rectangle (3..5 wide/tall, effort 2 or 3) around j^*. Naturally explores in proportion to posterior probability — good when the posterior is multimodal or you fear lock-in.

5. Simulator and multi-campaign benchmark

SearchEnvironment mirrors the search-missions Streamlit API exactly: it accepts (x_min, x_max, y_min, y_max, effort), enforces cost = effort · |R|, debits the budget, and returns $s_t \in \{0, 1\}$. The true detector (TrueDetector) is a deliberately different ρ-shape than the one the inference uses, so the simulation does not auto-justify the modelling choice.

Multi-campaign benchmark. For each of K campaigns we sample a single truth cell $z_k \sim \pi$ and replay every selected strategy on the same z_k (paired design — much lower variance than independent samples). Truth changes across campaigns, so the aggregated detection rate estimates how often the strategy finds the object under the true sampling distribution.

The Streamlit app exposes this as a single button: pick the number of campaigns, the prior, the strategies, and the confidence threshold for commit_and_verify, then watch the progress bar; the summary table and bar charts appear automatically at the end. A paired per-campaign table makes it easy to see whether strategy A wins on the same truths where strategy B loses.

Mapping to the real submission

After choosing a strategy in the local simulator, the real deliverable workflow is: (1) compute the proposal rectangle from the current posterior, (2) enter it in the search-missions Streamlit form, (3) read back s_t, (4) append a MissionRecord with the same shape used locally, (5) call posterior_update on the new history. Because the local env and the webapp share the same API, no inference code changes.

6. Streamlit interface reference

Section	Purpose
Sidebar — Simulator	Seed, total budget, and sliders of the hidden TrueDetector (base, p_unit, alpha_d, alpha_r). F
Top metrics	Completed missions / Budget total / used / remaining — mirror of the webapp's Team status p
Prior	Pick one of the four priors and depth_bias. Changing this recomputes the posterior over the e
Detection model (ours)	Sliders for lambda_0, a_d, a_r of the inference model (separate from the hidden true detector
Truth	Plant the hidden cell from the displayed prior, uniformly at random, or by manual coordinates.
Mission preview	Central heatmap with selectable underlay (depth / prior / posterior); mission rectangles drawn
New mission	Webapp-style form with optional auto-fill from one of the five strategies. Shows Mission ID, Co
Detectability ρ and likelihood	Four heatmaps (ρ, q for the pending rectangle, L if s=0, L if s=1) plus two counterfactual poste
Bayesian update breakdown	Slider through every past mission; three heatmaps (prior before mission t, likelihood of missio
Multi-campaign benchmark	Run K full campaigns Monte-Carlo, paired across strategies. Summary table + bar charts + pe
Mission history	Full per-mission table plus a Download missions.csv button in the exact format requested by t

7. Discussion (Task 5)

Strengths

The model is fully probabilistic from end to end: every ingredient (prior, detection rate, likelihood) has a clean interpretation; posterior updates are exact and numerically stable. The four-prior comparison demonstrates that the physics + witness information dominates the uniform baseline by a wide margin. The five strategies cover the natural spectrum from pure exploitation (max_expected_detection) to pure exploration (thompson), with commit_and_verify giving a principled answer to the cool-down problem in the real webapp (few but more decisive missions).

Limitations

The biggest is **model misspecification**: our ρ assumes a particular functional form. If the real detector decays faster with depth than $(1 - d)^{a_d}$, our posterior will be over-confident after a single miss in deep cells and the strategy may move on too soon. commit_and_verify mitigates but does not eliminate this. We also assume conditional independence of detection events given Z — false if e.g. the sensor degrades after several passes.

What I would improve with more data

(i) Calibrate λ_0, a_d, a_r by fitting the model to the four previous mission reports — currently the parameters are chosen qualitatively. **(ii)** Replace the deterministic τ_{fall}, τ_{drift} in the prior with priors of their own, marginalising them out. **(iii)** Multi-step lookahead in the strategy (current info_gain is one-step myopic). **(iv)** Tighter use of the witness statements via a small Bayesian network rather than the current hand-tuned shifts.